multiGGM R package demo

Margaret Janiczek

2026-01-30

```
# load multiGGM package and other packages

library(multiGGM)

library(pheatmap)
set.seed(1)
```

Generate toy data

I generated some small toy data using the following settings:

- n = 100
- p = 50
- K = 5 groups

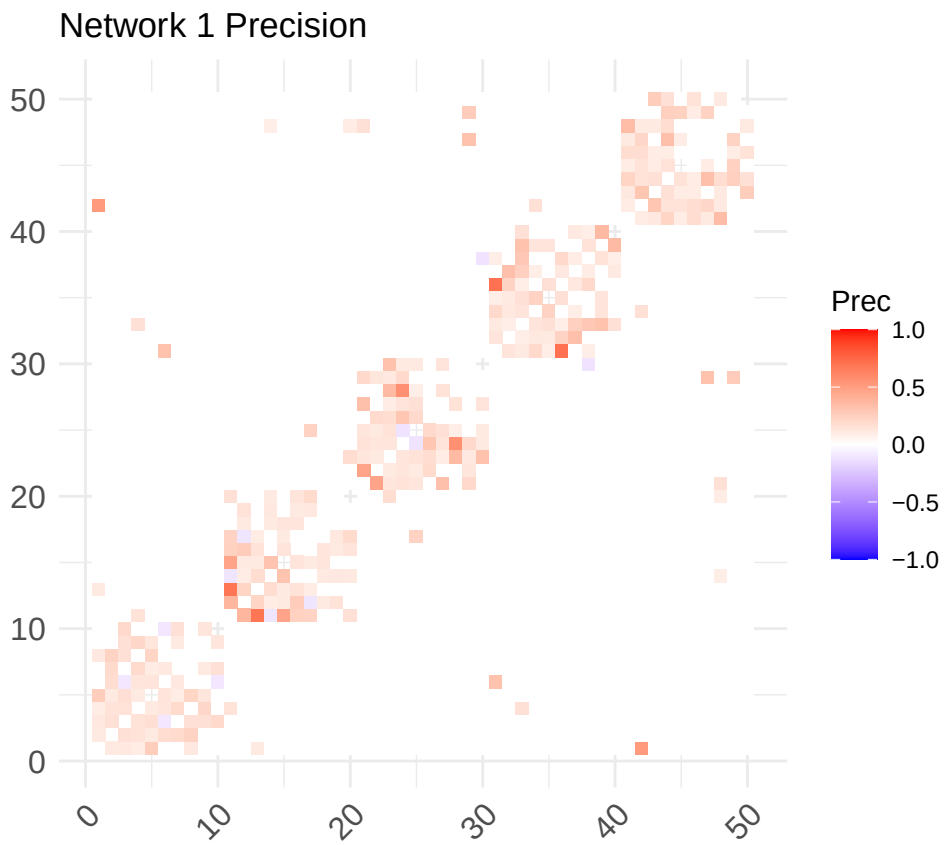
Within the 50 variables, for Network 1 I created an adjacency matrix by splitting the 50 variables into five groups, then generating moderately dense networks within those subblocks using scale-free network topology (Barabasi-Albert model). Then I generated a small number of between-block edges. Within block density was around 30%, between block density around 1%.

To weight the edges of the network I drew from an empirical distribution (generated from true network data from a metabolomic dataset). Edge weights ranged from abs(.095, 0.765), with smaller edge weights having more density in the distrubution to reflect a true biological setting where larger weights are less likely.

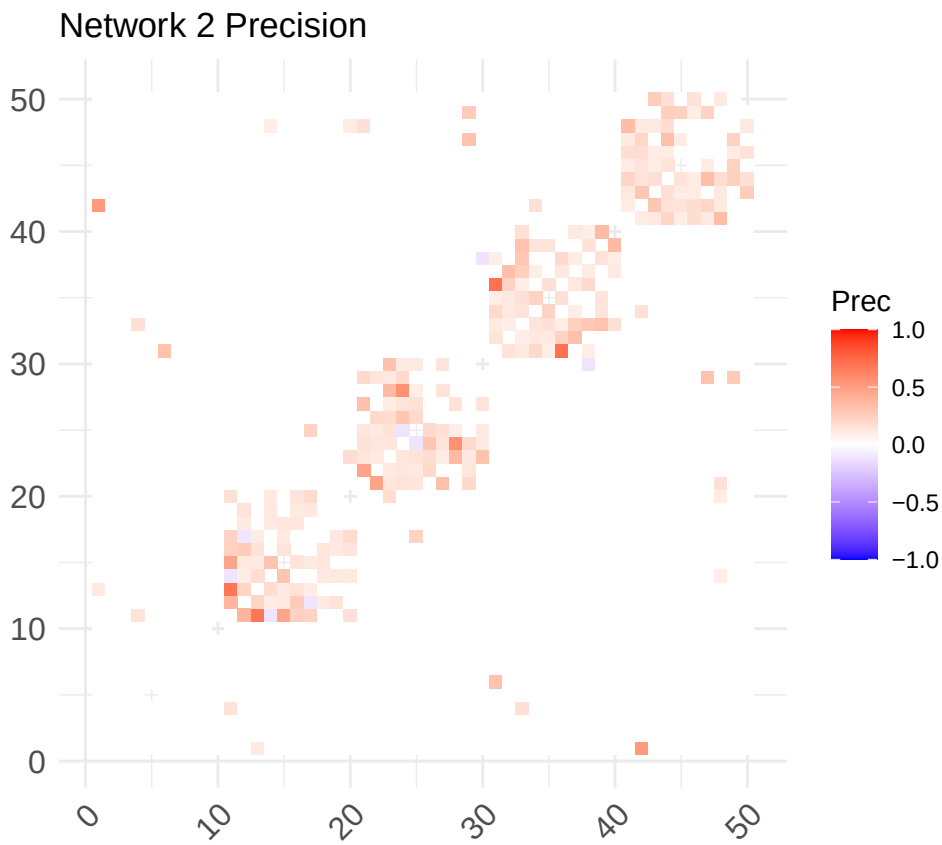
For this toy setting I am making the difference between the two networks prominant enough to detect. To generate Network 2, I deleted all within-block edges from one block, to emulate a biological setting where there is massive loss of function in a subnetwork. The edge weights are identical between the two networks in this example for all shared edges.

Below I visualize the precision matrices for each group.

```
# data simulated in dina.simulation project "simulated data demo"

# p = 50, n = 100, K = 2, 5 block structure. In Group 2 edges within
# one of the blocks is deleted to create diffential network
load(here::here("R/sim_dat_gold_p50.Rdata"))

#block_id <- c(rep(1, 10), rep(2, 10))

ggcorrplot::ggcorrplot(sim_gold$Omega$Omega1, show.diag = FALSE, outline.color = NULL, legend.title = "
```




```
ggcorrplot::ggcorrplot(sim_gold$Omega$Omega2, show.diag = FALSE, outline.color = NULL, legend.title = "
```



I ensure the weighted adjacency matrices are positive definite to make the precision matrices, then invert them to be the covariance matrices for each group, and use those to generate multivariate data using `mvnorm()`.

Finally, I calculated the sample covariance matrix from the generated data since in the in the original Matlab Code it takes the sample correlation matrix as input.

**Question: While matlab script specifies covariance matrix as input, it looks like in the code they just use X'X? Which should we use here?

```
p <- 50
n_vec <- c(100, 100)

# multiGGM takes sample covariance matrices as input
S_list <- list(cov(sim_dat$X1), cov(sim_dat$X2))

S_list2 <- list(t(sim_dat$X1)%*%sim_dat$X1, t(sim_dat$X2)%*%sim_dat$X2)

# # TODO formalize this somehow?
# ridge <- 1e-4
# S_list <- lapply(S_list, function(S) {
#   S <- 0.5 * (S + t(S))
#   S + ridge * diag(nrow(S))
# })
```

***Question for Christine/Grace: should input into the algorithm be sample covariance matrix or

Specify hyperparameters

In this example I used the same priors as in the matlab code except for w - the mixture weight between spike and slab (I think).

*** This section about hyperparameters I could use input on to explain what it all does and what will happen if you change things, and if you need to do tuning.

```
# Choose hyperparameters
# These are reasonable small-example defaults. Adjust as needed.

#Beta(1,1) would be flat/uninformative
#Beta(1,4) would be strictly decreasing
#Beta(2,5) would be unimodal skewed right
hyper <- list(
  b_prior = 3,                # b in G-Wishart prior
  D_prior = diag(p),          # D in G-Wishart prior
  a = 1, b = 4,               # Beta prior for edge probabilities (if used)
  alpha = 2, beta = 5,        # spike-slab / mixing priors (if used)
  w = 0.6              # mixture weight bw spike and slab
  ,
  alpha_prop = 1,
  beta_prop = 1
)
```

The main R function interface for the MCMC sampler is `multiggm_mcmc()`. After setting up the data and hyperparameters it's pretty easy to run the function:

```
fit_dbg <- multiggm_mcmc(
  S_list = S_list, n_vec = n_vec,
  burnin = 2000, nsave = 1000, thin = 2,
  nchains = 2, parallel = FALSE,# ncores = 2,
  seed = 123, hyper = hyper
)
```

This took 3540 seconds (about an hour) with 50 variables, 2000 burnin, 1000 save, thin 2 nchains 2.

```
# I ran  the below ahead of time and saved it to speed up knitting

# #updated the theta thing
# tictoc::tic()
# fit_dbg_50 <- multiggm_mcmc(
#   S_list = S_list, n_vec = n_vec,
#   burnin = 2000, nsave = 1000, thin = 2,
#   nchains = 2, parallel = FALSE,# ncores = 2,
#   seed = 123, hyper = hyper
# )
# tictoc::toc()



# this took about 303 seconds to run 20 variables with 2000 burnin nsave 1000 thin 2 nchains 2
# took 2028 seconds with 100 variables, 200 burnin 100 save one thin 1 chain
# took 174 seconds with 50 variables, 200 burnin 50 save thin 1 chain 1

# save(fit_dbg_100, file = "fit_demo_100.Rdata")
#save(fit_dbg_50, file = "fit_demo_50.Rdata")
```

```r
load("fit_demo_50.Rdata")
```

The resulting object is a list when contains the following elements:

```r
names(fit_dbg_50)
```

```
## [1] "chains"  "call"    "K"       "nchains"
```

And right now within the chains I have a bunch of objects save, many of them for debugging purposes so we can edit this later:

```r
str(fit_dbg_50$chains[[1]])
```

```
## List of 17
##  $ K             : int 2
##  $ p             : int 50
##  $ C_save        : num [1:50, 1:50, 1:2, 1:1000] 42.81 0 6.73 0 3.81 ...
##  $ Sig_save      : num [1:50, 1:50, 1:2, 1:1000] 0.02504 0 -0.00423 0 -0.0019 ...
##  $ adj_save      : num [1:50, 1:50, 1:2, 1:1000] 0 0 1 0 1 0 1 0 0 1 ...
##  $ Theta_save    : num [1:2, 1:2, 1:1000] 0 0 0 0 0 0 0 0 0 0 ...
##  $ nu_save       : num [1:50, 1:50, 1:1000] 0 -1.03 -2.61 -3.19 -2.25 ...
##  $ ar_gamma      : num [1:2, 1:2] 0 0 0.006 0
##  $ ar_theta      : num [1:2, 1:2] 0 0 0 0
##  $ ar_nu         : num [1:50, 1:50] 0 0 0 0 0 0 0 0 0 0 ...
##  $ toggle_attempt: int [1:2, 1:2] 0 0 4000 0
##  $ toggle_accept : int [1:2, 1:2] 0 0 24 0
##  $ theta_attempt : int [1:2, 1:2] 0 0 12 0
##  $ theta_accept  : int [1:2, 1:2] 0 0 0 0
##  $ dbg_theta     :List of 11
##   ..$ theta_prop_pos       : int 3988
##   ..$ logar_finite         : int 4000
##   ..$ logar_gt0            : int 12
##   ..$ accept               : int 24
##   ..$ dbg_toggle_up_attempt: int 3988
##   ..$ dbg_toggle_up_accept : int 12
##   ..$ dbg_toggle_dn_attempt: int 12
##   ..$ dbg_toggle_dn_accept : int 12
##   ..$ dbg_logar_up_gt0     : int 0
##   ..$ dbg_logar_dn_gt0     : int 12
##   ..$ dbg_w                : int 4820516
##  $ hyper         :List of 9
##   ..$ alpha     : num 2
##   ..$ beta      : num 5
##   ..$ a         : num 1
##   ..$ b         : num 4
##   ..$ my_w      : num 0.6
##   ..$ alpha_prop: num 1
##   ..$ beta_prop : num 1
##   ..$ a_prop    : num 2
##   ..$ b_prop    : num 4
##  $ call          : language .multiggm_mcmc_single(S_list = S_list, n_vec = n_vec, burnin = burnin,
##  - attr(*, "class")= chr "multiggm_fit"
```

But for now let's assume this is okay and move on to some other things we can look at.

After fitting multiGGM

1) Confirm both chains are valid fit objects
2) Summarize acceptance diagnostics (nu, gamma, theta)
3) Quick convergence sanity check (Theta trace and edge density)

A. Theta trace (single parameter when K=2)

B. Graph sparsity trace (per group)

4) Extract edge posterior probabilities (PPIs) and compare chains

Let's take a look at some of the diagnostic outputs. This is the part that is showing some of the problems I'm having - excess of zeros and the alg seems to be "stuck" in the spike. I've tried multiple different combinations of hyperparameters without really being able to improve much.

```
f <- fit_dbg_50$chains[[1]]

f$ar_gamma
```

```
##      [,1]  [,2]
## [1,]    0 0.006
## [2,]    0 0.000
```

```
f$ar_theta
```

```
##      [,1] [,2]
## [1,]    0    0
## [2,]    0    0
```

```
f$ar_nu[1:5, 1:5]
```

```
##      [,1]    [,2]    [,3]    [,4]    [,5]
## [1,]    0 0.38575 0.84225 0.38275 0.84350
## [2,]    0 0.00000 0.36825 0.35450 0.36075
## [3,]    0 0.00000 0.00000 0.38600 0.34500
## [4,]    0 0.00000 0.00000 0.00000 0.39175
## [5,]    0 0.00000 0.00000 0.00000 0.00000
```

```
#f$dbg_theta

#
# c(
#   toggle = f$toggle_accept[1,2]/f$toggle_attempt[1,2],
#   within = if (f$theta_attempt[1,2] > 0) f$theta_accept[1,2]/f$theta_attempt[1,2] else NA_real_,
#   prop_zero = mean(f$Theta_save[1,2,]==0),
#   slab_mean = mean(f$Theta_save[1,2,][f$Theta_save[1,2,]>0]),
#   slab_sd   = sd(f$Theta_save[1,2,][f$Theta_save[1,2,]>0])
# )
```

Right now the alg is only identifying 9 non-zero edges, which is far less than the data I generated.

So maybe there are some easy quick fixes, or maybe not, but I hope I can get it to work in R!!

```
#Posterior summaries: edge probabilities + partial r



ppi <- apply(f$adj_save, c(1,2,3), mean, na.rm = TRUE)
```

```r
test <- ppi[,,1]

total_edges <- sum(test[upper.tri(test)])
total_density <- total_edges / (p * (p - 1) / 2)

total_edges
```

```
## [1] 9
```

```r
total_density
```

```
## [1] 0.007346939
```